

Transmodel: choosing tolerances

Discussion notes for Sariel Ye | 23 September 2026

Recommendation: keep the 0.1, 0.01 and 0.001 levels as evaluation milestones. Initially use the same physical tolerances across the domain, but report success separately for different motion conditions. A single overall score would hide too much.

This discussion is based on `transmodel_training_plan.md`, the current physics and network code, and the 1,000-example validation set. No model has been trained for this analysis. The evidence concerns the target function and evaluation design.

1. The discontinuity is real; the ‘4’ is a force

The current simulator's static-friction threshold is a force, not a minimum velocity of approximately 4 m/s:

$$F_s = \mu_s mg = 0.4 \times 1 \times 9.81 = 3.924 \text{ N}$$

Immediately above this threshold, kinetic friction applies. The initial acceleration approaches:

$$a = (\mu_s - \mu_k)g = 0.981 \text{ m/s}^2$$

For an initially resting cuboid, the reference code gives the following outcomes. Here t_1 is the push duration and t is the observation time.

Force	t_1	t	Displacement	Final velocity
At threshold	1 s	1 s	0 m	0 m/s
Threshold + 0.000001 N	1 s	1 s	$\approx 0.4905 \text{ m}$	$\approx 0.981 \text{ m/s}$
Threshold + 0.000001 N	1 s	2 s	$\approx 0.654 \text{ m}$	0 m/s
Threshold + 0.000001 N	0.1 s	0.1 s	$\approx 0.004905 \text{ m}$	$\approx 0.0981 \text{ m/s}$

An arbitrarily small force change can therefore produce a finite output change. **The size depends on the time inputs.** Coasting can eliminate the final-velocity difference while preserving the displacement difference.

This is a discontinuity in the mapping from inputs to future outcomes. Within each individual trajectory, velocity still changes continuously over time; static friction does not cause an instantaneous velocity jump.

2. What this means for the current network

Your linear layers and ReLU activations form a continuous function. A continuous predictor cannot uniformly approximate a discontinuous target to arbitrarily small error across both sides of the boundary.

For a target jump of size J , the supremum of the absolute prediction error around the boundary must be at least $J/2$. In the one-second example, the bounds are approximately **0.24525 m** for displacement and **0.4905 m/s** for velocity.

Consequently, even the 0.1 level cannot be guaranteed for **every possible input** by this continuous network under the current idealized physics. A finite dataset can still show high accuracy: the network can approximate the jump with a steep transition, concentrating failures in a narrow region.

This evidence challenges a universal guarantee and the model's representation. It does not, by itself, establish that acceptable physical error should be larger near friction boundaries.

3. Use three levels as evaluation milestones

Define separate errors and tolerances for the two outputs:

$$e_d = |\text{predicted } d - \text{reference } d| \quad e_v = |\text{predicted } v - \text{reference } v|$$

A sample passes only when **both** $e_d \leq \epsilon_d$ and $e_v \leq \epsilon_v$.

Milestone	Displacement tolerance	Velocity tolerance
Coarse	0.1 m	0.1 m/s
Intermediate	0.01 m	0.01 m/s
Fine	0.001 m	0.001 m/s

These are provisional experimental targets. Equal numerical values for metres and metres per second are a convenience, not a physical requirement.

Avoid calling the first milestone ‘the model generally learns.’ A model may improve substantially without reaching it, or pass many easy examples while missing important behavior. Report something concrete: ‘95% of validation examples meet the coarse joint tolerance.’

Evaluate all three levels at every validation check, using the same training loss. There is no need to switch the training objective whenever a level passes.

4. A small tolerance system for this project

Physical accuracy. Report the joint pass rate at all three levels, plus displacement and velocity mean absolute error (MAE), 95th-percentile absolute error, and maximum observed error. The 95th percentile is the error at or below which 95% of evaluated examples fall.

Condition-specific accuracy. Calculate pass rates separately for always-resting cases, cases that move and then stop, and cases still moving at observation. Also report a dedicated boundary group.

Coverage. Always show the number of examples in each group. A percentage from five examples gives very little evidence.

Start the boundary group with initially resting inputs satisfying:

$$||F| - F_s| \leq 0.2 N$$

This matches an existing sampling band. It is a diagnostic choice, not a physical constant; freeze it across comparisons. Later, add fixed slices approaching the threshold from both sides and approaching $v_0 = 0$, since a broad band can conceal a narrow failure region.

For a first experiment, **95% overall and 95% within each main motion group** is a reasonable candidate milestone. These percentages are choices to investigate, not established application requirements. Show boundary performance explicitly; if it falls short, report that limitation rather than quietly relaxing its tolerance.

The existing validation data demonstrates why grouping matters: **481 of 1,000 examples have zero final velocity**, but only **64 remain stationary throughout**. Velocity accuracy alone could look encouraging while displacement predictions remain poor.

5. When should tolerances depend on the case?

Postpone condition-dependent tolerances until you can justify them through the intended use. A larger tolerance near breakaway would make the benchmark easier exactly where the model struggles. That is justified only if those errors are genuinely less consequential.

Relative error alone is unsuitable: it is undefined at zero and unstable near zero. If future applications care about proportional accuracy over large distances, use a mixed rule such as:

$$|\text{predicted } d - \text{reference } d| \leq a_d + r_d |\text{reference } d|$$

Here a_d is an absolute allowance in metres and r_d is a dimensionless relative allowance. Define a separate velocity rule. The absolute term handles values near zero. For now, fixed absolute tolerances and separate reporting are simpler.

6. Other risks to keep visible

Tiny velocity errors can change the physical regime. In the current reference, $F = 3.5 \text{ N}$ and $v_0 = 0$ produce rest. With $v_0 = 10^{-8} \text{ m/s}$ and a one-second push, the result is approximately $d = 0.2785 \text{ m}$ and $v = 0.557 \text{ m/s}$. This makes feeding predictions back into later steps particularly sensitive.

Endpoint accuracy can hide incorrect behavior. A cuboid that moved and stopped has $v = 0$, just like one that never moved. Predicting both outputs helps, but later trajectory checks still matter.

Single-prediction tolerances do not guarantee rollout accuracy. Errors accumulate and can trigger different friction branches. Repeated predictions need separate position and velocity targets at specified horizons.

Millimetre agreement with the simulator is not millimetre accuracy in reality. Friction uncertainty and the ideal static/kinetic switch may dominate real measurements.

Rare failures remain uncertain with small evaluation groups. Repeated validation checks do not provide new independent evidence; keep the test set reserved.

7. Suggested next experiment

Run the existing baseline and evaluate all three tolerance levels with the condition groups above. If failures concentrate at the discontinuities, that becomes evidence for targeted data or an explicit rest/sliding decision in the model. It would not establish that the whole project needs more labels or looser tolerances.

The useful result is a clear account of where the predictor works, where it fails, and whether those failures come from data coverage, training, or the continuous model's inability to reproduce the idealized switch.

Project sources and scope

transmodel_training_plan.md
cuboid_translation_x/physics_formula.py
cuboid_translation_x/neural_network.py
labels/README.md
labels/validation.csv

The numerical boundary examples were checked against the current reference function. Validation group counts were calculated from the existing validation CSV. These are reference and dataset checks, not results from a trained model. The training plan and source code were not changed for this PDF.